PROJECT-BASED LEARNING

PHASE-II EVALUATION

DNA (Destructured Neural Agent)

Evolutionary Autonomous Robotic Navigation Simulation
Using Genetic Algorithms in C++

Team ID: **DSCPP-III-2026-T051**

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T051
Semester: 3rd
Section: J, DS-2, Cyber, H
Domain: AI / Evolutionary Robotics

Team Members

1. Tanishka Mahar – 2510011047
2. Mansha Khati – 2510380049
3. Alankrit Juglan – 2510350082
4. Vinamra – 2510012765

Mentor

Ms. Megha Garg

Interactions completed: _____

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed a C++ robot simulation driven by a Genetic Algorithm. Delivered the problem definition, architecture and a working GA prototype (fitness, selection, crossover, mutation).

Feedback Received

- *Innovative idea*
- *Clarify the project explaination*
- *Add real-world application*

Changes Incorporated

- Rendering moved from SFML to Python
- Minimum share per individual; 5% mutation
- Headless C++ evolution loop

Problem Statement & Finalized Objectives

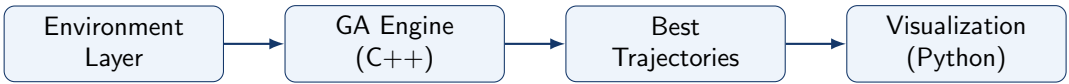
Problem Statement

Robots using fixed movement strategies fail in unfamiliar obstacles or changing environments. We need software-simulated adaptive strategies that improve through repeated trials, without manually defining every possible situation.

Finalized Objectives

1. Evolve robot movement strategies with a Genetic Algorithm (fitness, selection, crossover, mutation)
2. Simulate a virtual robot with a target and obstacles in a bounded 2D arena
3. Score strategies on distance to target, steps taken and collision penalties
4. Visualize the best trajectory per generation with live convergence metrics

Updated System Architecture / Workflow



Major Components

- Environment: arena, obstacles, target
- GA Engine: population, fitness, operators
- Visualization: trajectories, metrics

Changes Since Phase-I

- Visualization now in Python (was SFML)
- Python draws only the best paths
- Genes $\rightarrow$ (dx, dy) moves *(in progress)*

Module Design & Functional Requirements

Major Modules

1. Environment
2. Population & Fitness
3. Selection & Operators
4. Visualization & Metrics

Functional Requirements

- Generate 150 random strategies
- Score: distance, steps, collisions
- Roulette selection with elitism
- Crossover; 5% mutation
- Render best path per generation

Module Dependency

Fitness depends on the Environment; selection and operators consume fitness scores; Visualization reads only the GA's best trajectories.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++ core, Python visualization
- Database: Not required
- Framework: Python 2D graphics (library being finalized)
- Development Tool: Visual Studio Code
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Arrays: 2D population array, fitness tally, parent-share pool	GA Engine
OOPs with C++	Class hierarchies designed; STL (`std::vector`) refactor in progress	Robot & GA Engine
Operating Systems	Not required in current scope	N/A
DBMS	Not required; file storage if needed	N/A

Important: only concepts **actually implemented** are listed; planned work is marked in progress.

Development & Implementation Progress

Module	Status	Work Completed
GA Engine	**Completed**	150-member population, seeding, evaluation loop with convergence check
Fitness & Operators	**Completed**	Share-based roulette selection, single-point crossover, 5% mutation (string-target prototype)
Movement & Collision	**In Progress**	Mapping genes to (dx, dy) vectors and obstacle boundaries
Python Visualization	**In Progress**	Designing the 2D arena, trajectory and metrics display
STL Refactor & Integration	**Pending**	`std::vector` migration, C++–Python integration testing

Overall Project Completion: ∼50%

Working Prototype / Demonstration

Implemented Features

- Random population of 150 strategies
- Per-generation fitness scoring
- Roulette-wheel parent selection
- Single-point crossover, 5% mutation
- Auto-stop on full convergence

Prototype Output (C++ console run)

```
Target:  DNA Destructured Neural Agent
Gen 0     Fitness:  3/29
Gen 50    Fitness:  19/29
Gen 100   Fitness:  24/29
Gen 200   Fitness:  27/29
Gen 284   Fitness:  29/29
Best:  DNA Destructured Neural Agent
SOLVED in 284 generations!
```

Actual output from our prototype (29-character target).

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Population Init	150 random strings	150 individuals, all valid characters	Pass
Fitness Function	Score = positions matching target	Candidates scored correctly	Pass
Selection	Fitter candidates get larger share	Share grew with fitness	Pass
Crossover	Child takes parts of both parents	Valid segments, nothing lost	Pass
Mutation Rate	Random mutation, within 5% limit	Stayed within 5% probability	Pass
Convergence	Population converges; loop ends	Solved in 284 generations	Pass

Results / Validation

6 of 6 prototype tests passed. Collision and 2D movement tests will follow integration.

GitHub Repository & Development Activity

Repository Information

- https://github.com/Tanishka-ship-it/EvoWalk
- Active branch: main
- Dev: feature/python-visualization
- README.md: *updated*
- Contributors: *All 4 members added]*

Development Activity

- *Regular commits tracking development history*
- *Project code till now is uploaded*
- *Each member's contribution documented*

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Individual Contribution

Member	Role	Technical Contribution	Commits
Vinamra	Lead	System architecture, workflow design, C++ core interfaces and class hierarchies	*Done*
Alankrit Juglan	Developer	GA core prototype in C++: 150-member population, seeding, evaluation, crossover, mutation	*Done*
Mansha Khati	Algorithm	Fitness function, roulette-wheel share distribution, mutation probability set to 0.05	*Done*
Tanishka Mahar	Testing / Docs	Unit tests for initialization, crossover integrity, convergence stability; milestone documentation	*Done*

Individual Evaluation

Every member will explain and demonstrate their own contribution.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Premature convergence
- 1D genes to 2D movement
- Slow per-frame rendering

Solutions / Learning

- Minimum share; 5% mutation
- Headless C++; draw best paths only

Phase-III Roadmap

1. Gene $\rightarrow$ (dx, dy) and collisions
2. Python 2D arena and trajectories
3. `std::vector` refactor, integration
4. Testing and optimization
5. Final documentation and demo

Expected Outcomes & References

Expected Final Outcomes

- Working graphical simulation
- Fitness and convergence metrics
- Teaching tool for AI/robotics
- Lightweight, cross-platform core
- Future: OpenCV / IoT in Python

References

1. Genetic algorithm literature
2. C++ Standard Library docs
3. SFML documentation
4. Mutation breeding

Thank You

Questions & Suggestions